

Globala variabler

Variabler kan deklareras utanför en funktion (syntaxen är samma som för deklarationen av lokala variabler). Sådana variabler hör inte till någon speciell funktion, de kallas **globala**. Konventionen är att placera en deklaration av en global variabel upptill i filen. De kan initieras i deklarationen.

Det finns två kategorier av globala variabler, de som är globala för hela programmet och de som är globala endast för c-filen där de deklarerats. De senare skapar man genom att lägga till `static' före deklarationen.

För att kunna använda en program-global variabel måste dess namn finnas tillgänglig för kompilatorn. Detta är förstås inget problem då man använder variabeln i filen där den deklarerats, men hur blir det om en funktion i en annan c-fil vill komma åt en global variabel? - Jo, man måste exportera den i h-filen (analogt med funktioner). OBS, man ska då lägga till nyckelordet `extern' före deklarationen. Deklarationen av variabeln i en c-fil, även kallad definition, får bara finnas på ett ställe. Idén är ju att det är en och samma variabel som ska användas av olika delar i programmet. Några bra tumregler:

- Använd inte globala variabler om det inte behövs!
- Om du ändå behöver använda globala variabler så försök att arrangera så att de blir `static'.
- En definition av en global variabel (vare sig static eller ej) ska finnas på *ett* ställe.
- Om static så ska den *inte* exporteras (den är ju lokal för filen), annars *ska* den exporteras i h-filen (annars skulle vi ju gjort den static).

Funktioner och minnet.

En funktions data (dess lokala variabler och inparametrar) måste finnas på någon plats i minnet.

Denna plats bestäms inte av kompilatorn, plats för data reserveras under körning, noga räknat i det ögonblick då funktionen anropas.

Parametrarna fungerar som lokala variabler, med tillägget att de initieras till de värden som räknats fram från de uttryck som anges som anropsparametrar.

Exempel:

```
int bar(int a, int b)
{
    int x;
    x = a;
    return x+b;
}
void foo(int x)
{
    printf("%d", bar(x, x+42));
}
```

- Vid anrop av bar sker följande:
 - Plats allokeras för returvärdet (det som ska skickas till printf)
 - Plats allokeras för a_{bar} som ges värdet av x_{foo} .
 - Plats allokeras för b_{bar} som ges värdet av $x_{\text{foo}} + 42$.
 - bar anropas (noterar på stacken var man ska fortsätta efter nästa return).
 - Plats allokeras för x_{bar} .

Det minne som används är en stack, den s.k. systemstacken.

- När bar senare returnerar kommer värdena för a_{bar} , b_{bar} och x_{bar} inte längre att vara relevanta, platsen i minnet där de låg befinner sig då ovanför toppen på stackpekaren och är därför inte pålitliga.
- Kompilatorn hjälper oss till viss del att inte göra fel, om vi refererar till a, b eller x i foo så får vi fel för a och b. x tillåts men det är inte x_{bar} som avses, utan x_{foo} , så allt är frid och fröjd.

Ett annat exempel:

```
int bar(int a, int b)
{
    int x;
    x = a;
    return x+b;
}
int baz(int a)
{
    return bar(a) + 42;
}
void foo(int x)
{
    printf("%d", bar(x, baz(x)));
}
```

Vi ser i tabellen nedan (det är givetvis fiktiva adresser, men relationen mellan delarna är relevant) att de två anropen av bar får olika plats på stacken.

Stackkonfiguration i bar efter anropet i baz:

- [130] x_{bar} **<-stacktop**
-
- [124] a_{bar}
- [120] b_{bar}
-
- [110] a_{baz}
-
- [100] x_{foo}

Stackkonfiguration i bar efter anropet i foo:

- [120] x_{bar} **<-stacktop**
-
- [114] a_{bar}
- [110] b_{bar}
-
- [100] x_{foo}

Det fungerar enligt samma princip i Java, men i C har vi större behov av att reflektera över hur det egentligen funkar mer i detalj. Observera att det är den beskrivna principen som gör att det funkar med rekursion, varje nytt självanrop (rekursivt steg) som sker under exekveringen skapar precis som alla andra anrop en ny variabelomgivning för funktionens kod att arbeta i. Och samma teknik gör att man inte får göra några antaganden om att ett värde i en lokal variabel finns kvar från ett tidigare anrop.

Här är en skiss på hur det går till steg för steg från föreläsningen [[pdf-dokument](#)]. (Om du följer länken: i t.ex. "Acrobat reader" finns det ett val "Full screen" under "View" som du behöver använda för att se bildspelet på ett vettigt sätt)

I C finns det konstruktioner för det mesta, ofta också för att kringgå saker, så även i detta fall. Alltså: om man *vill* att ett värde i en funktion ska bibehållas mellan anropen så anger man **static** före i deklarationen, t.ex.

```
static int firsttime=1;
...
if (firsttime) {
    firsttime = 0;
```

```
... /* do some initialization stuff*/
}
```

Innebörden är att det finns exakt en variabel 'firsttime', den har en väldefinierad plats under hela programmets existens (den ligger inte på stacken). Man kan bara referera till den genom dess namn inifrån funktionen. Det kan ibland användas för hålla reda på något tillstånd, som i exemplet ovan.

Som ny kanske man ställer sig frågan sig om det är bra att ett språk innehåller så många möjligheter att komma runt saker, och möjligheter att göra "fula" saker som i C. Innan man dömer ut existensen av sådana konstruktioner så bör betänka att C är tänkt att användas bl.a. i tillämpningar där man behöver kunna göra "vad som helst". Jag menar att det därför är relevant att det ingår minst det som finns. Man kan säga att C är ett högnivåspråk på låg nivå.

*Detta ställer dock krav på oss som programmerare att inte missbruka sådana företeelser. T.ex. så är de globala variablerna i C helt oundgängliga då man jobbar med s.k. avbrottsrutiner. De kan även vara bra i andra sammanhang. **Men** om de används slentrianmässigt för att överföra information mellan funktioner kan man därmed göra ett program helt oläsbart och buggigt och framförallt gör man starka bindningar mellan olika funktioner, med påföljd att det blir hopplöst att underhålla och vidareutveckla programmet.*

Pekare

Pekarvariabler är variabler vars värden är avsedda för adresser till värden som finns någon annanstans, t.ex. en annan variabel.

Vid deklaration av en pekare anges vilken typ det ska vara på de värden som pekas ut.

Kompilatorn är noggrann med att vi sätter en pekare att peka på ett värde av fel typ. Det tillåts inte heller att vi blandar pekare och icke-pekare hur som helst. T.ex. får inte en floatpekare tilldelas värdet av en floatvariabel eller tvärt om.

Den är också noggrann med att vi inte blandar pekare till olika typer. T.ex. tillåts ej att vi tilldelar en doublepekare värdet av en int-pekare.

Det senare kan tyckas trångsynt: adress som adress kan det ju tyckas. Skälet är dock omtanke om oss; vi har med hög sannolikhet tänkt fel då vi försöker oss på en sådan tilldelning.

Vi inför två nya operatorer & som har betydelsen "adressen till", och * som i deklarationer har betydelsen "är en pekare till" och i uttryck har betydelsen "värdet som pekas ut". Exempel:

```
double a=3.14, b, *d;
```

/ Ovanstående betyder "d är en pekare till double-värde", eller mer exakt: d är en pekare vars innehåll tolkas som adressen till ett double-värde" (a och b är som vanligt double-variabler) */*

```
d = &a; /* Betyder "d tilldelas adressen till a" */
b = *d; /* Betyder "b tilldelas värdet som pekas ut av d" */
```

Mer...

```
int *p; p är en pekare till int. Värdet av p kan bara vara en adress till ett in
```

```
int i, j;
i = 13;
p = &i;
j = *p; j ges nu värdet som finns i variabeln i (dvs 13)
```

*p betyder värdet av den variabel (eller mer exakt: den plats) som p pekar ut eller det indirekta värdet av p (indirection används i boken). T.ex. ger j = i * 3; samma resultat som j = *p * 3;

```
void foo(int *p, int i)
{
    *p = i;
}

int main(void)
{
    int x;
    x = 3;
    foo(&x, 239);
    printf("%d", x); /* vad skrivs ut nu ??? */
    return 0;
}
```

Varning! Att avreferera en pekare som inte getts en giltig adress leder vanligen till krasch.

Exempel:

```
float *x, a=9.81;
*x = 0; /* Aj, aj, x har ett odefinierat värde, vi accessar här minne som vi i
x = &a;
*x = 0; /* Helt OK, nu sätter vi a:s värde till 0 */
```

Här kan du se steg för steg vad som händer med pekare vid funktionsanrop, välj form: [[OpenOffice presentation](#)] eller [[pdf-dokument](#)].

Är detta med pekare verkligen något som behövs? Kunde man inte t.ex. gjort två funktioner, en för vinkel och en annan för beloppet, då hade dessa kunnat *returnera* de värden de beräknat? - Svar på den senare frågan: Jovisst, det hade kanske t.o.m. varit en bättre design till det lilla problemet, men det var bara ett exempel... Svar på den första: Ja, detta är C, välkommen!

Just möjligheten att "returnera" värden till fler än en variabel är kanske inte den viktigaste användningen, det är snarare

- att ge en funktion möjlighet att komma åt en mängd värden hos anroparen (mer detaljer när vi kommer till arrayer och strukturer...) utan att skicka över dessa, utan istället bara skicka en pekare.
- möjligheten att kunna hantera dynamiskt allokerat minne...

Dynamiskt minne

Dynamiskt minne innebär att man kan ge programmet tillgång till en given mängd minne som vare sig är globala variabler, statiska variabler eller lokala variabler på stacken. Man ber helt enkelt "systemet" om att få tillgång till en skvätt minne. Om man får det (vilket man normalt får) så kommer detta minne att vara reserverat för programmets räkning till dess att man återlämnar det.

Syntaxen är:

```
int *p;
p = (int*)malloc(4);
if (p==NULL) {
    /* Felhantering */
} else {
    *p = 42; /* ok, vi har rätt till minnet som p pekar på */
    ....
}
```

Det betyder att p nu pekar på en "namnlös variabel". Men vet vi att storleken av en int är 4 byte? - Nej, i de labbmiljöer som rekommenderas är det så, men i allmänhet vet inte. Använd **alltid** sizeof-operatören istället för att göra som ovan. Vi får då:

```
int *p;
```

```
p = (int*)malloc(sizeof(int));
etc.
```

Varför är nu detta så bra? - Jo, det inses då vi kommit till arrayer och strukter som är lite större än bara de grundläggande typerna.

Dynamisk minne och felhantering

Det är som sagts tidigare av yttersta vikt att man inte "derefererar" en pekare innan den satts att peka på allokerat minne. Det är också viktigt att lämna tillbaks minnet då det inte längre behövs, senast innan programmet avslutas. De flesta operativsystem har en sådan minneshantering att då programmet avslutas så återtas allt minne från det. Vi ska ändå alltid lämna tillbaks det uttryckligen med anrop till free:

```
int *p, n;
...
p = malloc(sizeof(int) * n);
....
free(p);
```

Skälen är

- Vi visar oss själva att vi har ordning på minnet, det ser fullständigt och vackert ut
- Om man senare behöver göra ändringar och tillägg i programmet så kommer detta att vara svårare om man inte tydligt ser var dynamiskt minne allokeras, används, avallokeras (för att det inte behövs längre)
- Om man senare vill ta reda på om ett program verkligen har "minnesläckage" (dvs att man glömmer lämna tillbaks minne) så finns det verktyg som kan hjälpa en att göra detta. Om man struntar i att återlämna en del allokeringar som man (tror att man) vet alltid används under hela programmet och man därför egentligen inte skulle behöva lämna tillbaks så kommer verktygen inte att kunna särskilja dessa från verkligt minnesläckage.

Ok, nu dyker det upp en fälla till:

```
int *p, n;
...
/* Aja baja inte göra något med "*p" här */
....
p = malloc(sizeof(int) * n);
...
/* Här arbetar vi med "*p" och allt är ok */
....
free(p);
...
/* Återigen aja baja med "*p"! */
...
```

Lika viktigt som att inte accessa det som en oinitierad pekare pekar på är att inte accessa det efter att det är återlämnat. I bästa fall så kraschar det, i sämsta fall inte.

Om det ändå råkar "gå bra" utan krasch så är detta dåligt. Skälen till att det går bra att peta på ogiltigt minne kan t.ex. vara att det saknas minnesskydd (ren DOS eller olika inbyggda system) eller att detta är ofullständigt (DJGPP) så att man t.ex. tillåts att åtminstone läsa det. Det kan också i ett fullständigt minnesskyddat system vara så att en pekare råkar (av en slump) peka på ett stycke minne som har allokerats på ett korrekt sätt av någon annan programdel.

Gemensamt för alla orsaker är att vid ett annat körtillfälle kan slumpen ordna så att en oväntad krasch uppstår.

Faktum är:

- Ett mycket utbrett problem med C och C++ program i verkliga tillämpningar är just minnesläckage.
- Det vanligaste och mest problematiska minnesfelet är att man petar på avallokerat minne.

Slutsats: Noggrannhet!